Machine-Learning Interatomic Potentials in PyTorch

Part 2 — PyTorch foundations, taught through physics

*This is where the code starts. Everything here is taught in **one dimension** — a single distance `r` — on purpose: it strips away neighbor lists, periodic boxes, and 3D geometry so you can see the four PyTorch tools clearly. Those four tools (tensors, autograd, `nn.Module`, the training loop) are the entire machinery of an MLIP. In Part 4 we swap the 1D input for a real atomic environment and **almost nothing else changes.***

Companion files delivered with this part:

- `part2_foundations.py` — the complete, runnable, commented script. Run it: `python3 part2_foundations.py`.
- `part2_results.png` — the figure it produces (referenced in Section 5).

---

0. The one sentence to keep in view

> The network **is** the energy. The force is its derivative. Training teaches the energy to match data — and, crucially, teaches its *derivative* to match forces.

Part 1 gave you the physics of that sentence. Part 2 gives you the four PyTorch pieces that make it real:

Tool	What it is	Its job in an MLIP
Tensor	an array that also remembers how it was computed	holds positions, energies, forces, weights
Autograd	exact derivatives of any tensor computation	turns energy into **force** (`F = -dE/dr`), and drives learning
`nn.Module`	a container for parameters + a `forward` rule	*is* the energy function E_θ
Training loop	zero → forward → loss → backward → step	fits θ to your data

We'll take them in that order, then combine all four in a capstone that teaches a network the Morse potential and recovers its force.

---

1. Tensors — "NumPy arrays that remember how they were made"

You know NumPy, so 90% of this is free. A `torch.Tensor` behaves almost exactly like an `np.ndarray`:

```
import torch

x = torch.tensor([1.0, 2.0, 3.0])     # like np.array
z = torch.zeros(3, 4)                  # like np.zeros((3,4))
```

```
g = torch.linspace(0.75, 3.0, 400)     # like np.linspace
print(x.shape, x.dtype)                # torch.Size([3]) torch.float32
```

Everything you reach for in NumPy has a near-identical spelling:

NumPy	PyTorch
`np.array(...)`	`torch.tensor(...)`
`a + b`, `a * b`, `a @ b`	identical
`a.reshape(...)`, `a.sum()`, `a.mean()`	identical
broadcasting rules	identical
`a[a > 0]`, slicing	identical
`np.exp`, `np.sin`	`torch.exp`, `torch.sin`
`a.astype(np.float64)`	`a.double()` / `a.to(torch.float64)`

Interop is one call each way (they can even share memory):

```
import numpy as np
t = torch.from_numpy(np.array([1.0, 2.0]))   # numpy -> tensor
back = t.numpy()                              # tensor -> numpy
```

Three things that are genuinely **new** versus NumPy — and all three matter for MLIPs:

1. **`dtype` defaults to `float32`.** Fine for images; for *potentials* we usually want `float64`, because energy differences and their gradients need the precision (a wobbly last digit in E becomes a noticeable force error). The script sets this globally with one line:

   ```
   torch.set_default_dtype(torch.float64)
   ```

2. **`device`.** A tensor lives on CPU or GPU; `.to("cuda")` moves it. Training real MLIPs happens on a GPU. We stay on CPU here because 1D is tiny — but the code is identical; you'd just move the model and data to the GPU.

3. **`requires_grad`.** This is the big one, and it's the whole reason we're using PyTorch instead of NumPy. Set it on a tensor and PyTorch starts **recording every operation** you do to it, building a graph it can later differentiate. That recording is what Section 2 is about.

```
r = torch.tensor([1.2], requires_grad=True)   # "track me — I'll want dE/dr later"
```

That's the mental model: **a tensor is a NumPy array that, if asked, remembers the chain of operations that produced it — so PyTorch can run the chain rule backward through it.**

2. Autograd — exact derivatives, and therefore forces

This is the heart of the whole course. Take it slowly.

2.1 The idea

When you compute with tensors that `requires_grad`, PyTorch silently builds a **computational graph**: a record of every elementary operation (`+`, `*`, `exp`, matmul, …). Because it knows the derivative of each elementary operation, it can apply the **chain rule** backward through the whole graph to get the exact derivative of your final number with respect to any input. This is *automatic differentiation* — not finite differences (no small-step error), not symbolic algebra (no giant formula), and it costs about the same as computing the function once.

2.2 Two ways to ask for a gradient

Say `E` is a scalar you computed from `r`. There are two spellings; you'll use both.

(a) `.backward()` then read `.grad` — fills in the `.grad` field of the inputs:

```
r = torch.tensor([1.2], requires_grad=True)
E = 4.0 * (r**-12 - r**-6)     # a scalar (Lennard-Jones)
E.backward()                    # walk the chain rule backward
print(r.grad)                   # dE/dr, stored on r
```

The Theory of `backward()`

What `r` is — two layers. Physically, `r` is just the interatomic distance, `1.2 Å`; you know that part. The thing worth understanding is what `r` is as a PyTorch object. It's a tensor — a container of numbers, like a `NumPy` array — but created with `requires_grad=True`, and that flag changes everything. It tells PyTorch: "I care about derivatives with respect to this tensor, so from now on, every operation that uses r, write it down."

Two properties follow, both visible in the demo. `r.is_leaf` is True: `r` is a `leaf`, meaning we created it directly as an input, not as the result of some calculation. Leaves are the things you're allowed to differentiate with respect to — the "knobs." And `r.grad` starts as None: it's an empty slot that will later hold `dE/dr`. So `r` is simultaneously "the number `1.2`" and "a knob `PyTorch` is watching, with an empty drawer `(.grad)` waiting for its derivative."

The computational graph. When you then write `E = 4.0 * (r**-12 - r**-6)`, PyTorch does two things at once. It computes the number `(E = -0.891)`, and it silently records a graph of how it got there — the chain of elementary operations, each one knowing its inputs. In the demo you can see the tail end of that graph: `E.grad_fn` is `MulBackward` (the last operation was the `4.0 * multiply`), and walking back from it we find a `SubBackward` (the subtraction), whose two parents are the two PowBackward nodes (the `r**-12 and r**-6` powers). So the recording looks like this, built during the forward pass:

```
r ──► r**-12 ─┐
              ├─►  (a - b)  ──►  4*(…)  ──►  E
```

```
r ──► r**-6  ─┘
```

The crucial part: PyTorch knows the derivative rule for each elementary box (the derivative of a power, of a subtraction, of a scalar multiply). That's all it needs.

How `E.backward()` works — this is the whole thing. backward() computes `dE/dr` by running that graph in reverse and applying the chain rule. The chain rule says the derivative of a composition is the product of the local derivatives along the way; backpropagation is just that rule applied mechanically, node by node, from E back to r.

It starts by seeding the derivative of the output with respect to itself: `dE/dE = 1`. Then it steps backward through each box, at every step multiplying by that box's local derivative. Concretely, with the numbers at `r = 1.2`:

```
seed:                       dE/dE = 1
through  E = 4·c :          dE/dc = 4
through  c = a − b :        dE/da = 4·(+1) = 4      dE/db = 4·(−1) = −4
through  a = r^-12 :        da/dr = −12·r^-13 = −1.1216   →  contributes
4·(−1.1216) = −4.486
through  b = r^-6  :        db/dr =  −6·r^-7  = −1.6745   →  contributes
(−4)·(−1.6745) = +6.698
```

Now the key move: r feeds into two branches (it appears in both `r**-12` and `r**-6`), so its total derivative is the sum of the contributions coming back along both paths:

`dE/dr = −4.486 + 6.698 = +2.2117`

That's exactly the `r.grad = 2.211693…` the demo printed, and it matches the by-hand derivative $-48 \cdot r^{-13} + 24 \cdot r^{-7}$ to the last digit. PyTorch didn't do algebra or finite differences — it just multiplied local derivatives backward through the graph and summed over paths. `(That "sum over paths" is also, incidentally, why .grad accumulates and why you must zero_grad() between training steps — it's the same summing behavior, from Problem 5.)`

One physical footnote so the sign isn't mysterious: `dE/dr = +2.2117` is positive because `1.2 Å` sits just past the LJ minimum `(≈1.122 Å)`, so pulling the atoms apart raises the energy. The force is `−dE/dr`, i.e. it points to pull them back together — an attraction of magnitude `2.2117 eV/Å`.

The code, line by line

```
r = torch.tensor([1.2], requires_grad=True)    # 1
E = 4.0 * (r**-12 - r**-6)                     # 2
E.backward()                                   # 3
print(r.grad)                                  # 4
```

#Line1 creates the leaf tensor `r = 1.2` and flags it differentiable. At this instant `r.grad` is None.

#Line2 is the forward pass. Python computes the LJ energy `(-0.891)`, and because r requires grad, PyTorch simultaneously records the graph and hangs a grad_fn on E. Nothing is differentiated yet — the graph is just built and waiting.

#Line3 is the backward pass. It seeds `dE/dE = 1` and propagates backward through the recorded graph, chain-ruling its way to every leaf, and deposits `dE/dr` into `r.grad`. (Requirement: `E` must be a single scalar for bare `.backward()` — here it is. If `E` were a vector you'd either sum it first or tell backward how to weight the components.)

#Line4 reads the drawer that was empty before: `r.grad` now holds 2.2117.

Two connections to what you've already seen. The MLIP code usually uses the functional form `torch.autograd.grad(E, r)` instead — it returns `dE/dr` directly rather than stashing it in `.grad`, which is cleaner when you immediately want force = - that. And when `r` is not one number but a column of many distances `(M, 1)`, the exact same machinery runs; you differentiate `E.sum()` so that each sample's derivative lands in its own row (that was the trick in Problem 7).

So the one-sentence version: `r` is a tensor you've flagged as a differentiation knob, and `E.backward()` walks the recorded graph backward with the chain rule to fill `r.grad` with `dE/dr`. Everything else in MLIPs — forces, force-training, Hessians — is this same move, just with a network in place of the LJ formula.

(b) **`torch.autograd.grad(output, inputs)`** — returns the gradient directly, without touching `.grad`. This is the form MLIP code uses, because forces are cleaner as a returned value:

```
r = torch.tensor([1.2], requires_grad=True)
E = 4.0 * (r**-12 - r**-6)
(dEdr,) = torch.autograd.grad(E, r)     # returns a tuple
force = -dEdr                            # F = -dE/dr
```

That `force = -dEdr` line is the entire force calculation of an MLIP. Whether `E` comes from a one-line formula (here) or a 10-million-parameter network (Part 4), this line is **unchanged**.

2.3 Forces on many atoms at once

For N atoms, positions are an `(N, 3)` tensor and the force is `(N, 3)` too. Same call:

```
positions = torch.tensor([[0.,0.,0.],[1.2,0.,0.],[0.,1.1,0.]], requires_grad=True)
E = total_energy(positions)                      # your model: (N,3) -> scalar
forces = -torch.autograd.grad(E, positions)[0]    # (N,3), exact
```

One scalar in, a full force array out, one call. (In the capstone we use a batch trick — differentiate `E.sum()` — to get the derivative for many independent samples at once; it's commented in the script.)

2.4 The gotcha that explains `zero_grad()`

`.grad` **accumulates** — each `.backward()` *adds* to whatever is already there. That's deliberate (it enables some advanced tricks), but it means if you don't clear it between training steps, gradients from step 1 pollute

step 2. Hence every training loop starts by zeroing the gradients (`optimizer.zero_grad()`). File that away; it explains a line you'll see in Section 4 that otherwise looks like a ritual.

2.5 `create_graph=True` — the flag that lets you *train* on forces

Here is the subtlety Part 1 promised. The force is `F = -dE/dr`, a **first** derivative. To train on forces, the loss contains F, and to update the weights we differentiate the loss — so we differentiate F — with respect to the weights. That's a derivative **of a derivative**: second order.

By default PyTorch throws away the graph after computing a gradient (to save memory). To differentiate *through* a gradient, you must tell it to keep that graph:

```
r = torch.tensor([1.2], requires_grad=True)
E = 4.0 * (r**-12 - r**-6)
(dEdr,) = torch.autograd.grad(E, r, create_graph=True)   # keep the graph!
(d2Edr2,) = torch.autograd.grad(dEdr, r)                 # now a 2nd derivative is
possible
print(dEdr, d2Edr2)
```

- **Evaluating** forces (running MD, making a prediction): `create_graph=False` (the default) — faster, less memory.
- **Training** on forces: `create_graph=True` — so `loss.backward()` can reach the weights *through* the force computation.

Forget it during training and you'll get an error or zero gradients on the force term. Remember it and force-matching just works. That's the whole story of that flag.

2.6 Turning autograd *off*

For pure evaluation where you don't need any gradient, wrap the code in `torch.no_grad()` (skips graph-building, saves time/memory), or call `.detach()` to peel a tensor off the graph:

```
with torch.no_grad():
    E = model(r)          # no graph built; just the number
val = E.detach().numpy()  # a plain array, no autograd attached
```

(Note: to get *forces* you obviously can't be inside `no_grad` for the position derivative — but once you have the forces you `.detach()` them for logging/plotting.)

Section 2 in one line: autograd gives you exact derivatives; `-grad(E, positions)` is your force; `create_graph=True` is what makes *training on forces* possible; `zero_grad` exists because `.grad` accumulates.

---

3. `nn.Module` — packaging the energy function

A one-line formula doesn't need managing. A network with thousands of weights does — you need something to hold the parameters, move them to the GPU, save them, and hand them to the optimizer. That

container is `nn.Module`.

You subclass it, create your layers in `__init__`, and define the computation in `forward`. PyTorch then auto-discovers every parameter inside.

```
import torch.nn as nn

class EnergyMLP(nn.Module):
    def __init__(self, width=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(1, width), nn.SiLU(),      # 1 input (r) -> width
            nn.Linear(width, width), nn.SiLU(),
            nn.Linear(width, 1),                 # -> 1 output (energy)
        )
    def forward(self, r):
        return self.net(r)         # shape (M, 1)

model = EnergyMLP()
print(sum(p.numel() for p in model.parameters()), "parameters")
```

Three things worth naming:

- **`nn.Linear(a, b)`** is exactly `y = x W? + b` with learnable `W` (shape `b×a`) and bias — a matrix multiply. Stack a few with nonlinearities between them and you have a multilayer perceptron (MLP), a universal function approximator.
- **The activation must be smooth. This is physics, not taste.** We use **`nn.SiLU`** (also called swish, a smooth curve). Do **not** use `ReLU` here: ReLU has a kink, so its derivative jumps, so your *force* would be discontinuous — and discontinuous forces make MD explode and energy stop being conserved. Smooth energy ⇒ smooth force. Tanh, SiLU, GELU, softplus are all fine; ReLU is not.
- **`model.parameters()`** is the auto-collected list of all weights and biases. You hand exactly this to the optimizer in the next section. `model.to("cuda")` moves them all; `torch.save(model.state_dict(), ...)` saves them.

So `model` here *is* the function `E_θ(r)`. Calling `model(r)` runs the forward pass; `θ` are the numbers `parameters()` returns.

---

4. The training loop — the five lines you'll use forever

Learning = adjust θ to shrink a loss. Two ingredients:

- **A loss**: how wrong are we? Mean-squared error, `((pred - target)**2).mean()`.
- **An optimizer**: how do we use the gradient to improve θ? We use **Adam**, a robust default. You give it `model.parameters()` and a learning rate.

```
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
```

Then the loop. Memorize this shape — every PyTorch model you ever train has this skeleton:

```
for epoch in range(4000):
    opt.zero_grad()                      # 1. clear old gradients (.grad
accumulates!)
    E_pred = model(r_train)              # 2. FORWARD: predict
    loss = ((E_pred - E_train)**2).mean() # 3. how wrong?
    loss.backward()                      # 4. BACKWARD: d(loss)/d(weights)
    opt.step()                           # 5. nudge every weight downhill
```

Read it as a sentence: *clear the slate, make a prediction, measure the error, find which way each weight should move, take a step.* Repeat until the loss stops falling.

Turning this into an **MLIP** trainer is a one-line change — add the force term to step 3:

```
    E_pred = model(r_train)
    loss = ((E_pred - E_train)**2).mean()
    F_pred = forces_from(model, r_train)              # uses create_graph=True
inside
    loss = loss + ((F_pred - F_train)**2).mean()      # <-- force-matching
    loss.backward()
```

That single added term is the difference between "a curve fitter" and "a machine-learning interatomic potential." Everything else — data pipeline, optimizer, loop — is shared with all of deep learning.

(Two practical touches used in the script: `torch.nn.utils.clip_grad_norm_(model.parameters(), 10.0)` after `backward()` tames the occasional Adam spike, and for real problems you'd add a learning-rate scheduler and a validation-based early stop. We'll layer those in when they matter.)

---

5. Capstone — teach a network the Morse potential, then read off its force

Now all four tools together, doing something real. This is `part2_foundations.py`; here is what it does and what it proves.

Setup. The **Morse potential** `E(r) = D?(1 − e)² − D?`, with `e = exp(−a(r − r?))`, is our ground truth — it plays the role DFT plays in a real project (we know its energy *and* its exact force, so we can grade ourselves). We give the model only **18 training points** and then test on **400 unseen points**. Few points on purpose: it exposes the difference between fitting energy and fitting force.

Two models, identical except for the loss:

- `model_E` trains on **energy only**.
- `model_F` trains on **energy + force** (the one-line change from Section 4).

Both start from the *same* random initialization, so the comparison is fair.

The verified result (printed by the script; I ran it):

```
Generalization on 400 unseen points (trained on only 18):
model                      energy MSE      force MAE
energy-only                   0.00016        0.10608
energy + force                0.00001        0.01465
force MAE improved 7.2x by adding force-matching
```

Look at what that says. The energy-only model fits the **energy** beautifully (tiny MSE) — but its **force** (the derivative of that energy) is **7× worse** than the model that was also shown forces. The figure `part2_results.png` makes it visual, in four panels:

- **Top-left (Energy).** Both learned curves lie right on the true Morse curve. Energy is the "easy" target — 18 points pin it down.
- **Top-right (Force, full range).** Autograd recovers the entire force curve — the steep repulsive rise, the zero-crossing at the bottom of the well, the attractive tail — from a model that only ever output *energy*. This is the payoff of "force = −dE/dr for free."
- **Bottom-left (Force, zoomed).** Zoom into the chemically relevant region and the energy-only force (blue) visibly *wanders* around the truth, while the force-matched force (orange) hugs it.
- **Bottom-right (Force error, log scale).** The energy-only error curve sits *above* the force-matched one almost everywhere — the 7.2× gap, made concrete.

Why this is the most important lesson in MLIP training. A good energy fit does **not** guarantee good forces, and forces are what MD actually integrates. Because each configuration gives you 3N force numbers but only 1 energy number (Part 1 §1), **force-matching is where most of your training signal lives** — and it directly regularizes the derivative you care about. Every serious MLIP is trained on forces for exactly the reason you just saw in 1D.

Open the script, change things, and watch the table move: try `w_force=0.0` vs `10.0`, swap `nn.SiLU` for `nn.ReLU` (watch the force get jagged), drop `n_train` to 8, or switch `create_graph=True` to `False` in the force loss (watch it break). That is the fastest way to build intuition.

---

6. What you now own — and Part 3

You can now read and write every line of a PyTorch model: tensors carry the data, autograd turns energy into force and enables force-training (`create_graph=True`), `nn.Module` holds the energy function, and the five-line loop fits it. In 1D, you have already built a working — if tiny — interatomic potential.

The only thing separating this from a **real** MLIP is the input. Right now the model eats a single distance `r`. A real system is many atoms in 3D (and often a periodic box), and we cannot feed raw coordinates — Part 1 §2 forbids it (translation/rotation/permutation). So:

Part 3 — From atoms to model inputs. Neighbor lists within a cutoff, the minimum-image convention for periodic cells, the smooth cutoff function, and hand-built **invariant descriptors** (Behler–Parrinello symmetry functions) that turn each atom's messy 3D neighborhood into a fixed-length vector the network can eat — *without breaking a single symmetry*. That descriptor vector replaces the lone `r` of Part 2, and the MLP + autograd + training loop you already understand carry straight over.

Then **Part 4** assembles it into a complete, trainable, from-scratch potential on a real toy system.

---

7. Check yourself before Part 3

1. Name the one capability a `torch.Tensor` has that an `np.ndarray` doesn't, and why MLIPs need it. (§1)
2. What does `requires_grad=True` actually cause PyTorch to do? (§2.1)
3. Write the line that computes forces from an energy `E` and positions `pos`. (§2.2–2.3)
4. Why must a training loop call `zero_grad()` every step? (§2.4)
5. When do you need `create_graph=True`, and when should you *not* use it? (§2.5)
6. Why is `SiLU` acceptable but `ReLU` a bug in an energy model? (§3)
7. Recite the five-line training loop from memory, and say what each line does. (§4)
8. The capstone's energy-only model had tiny energy error but poor forces. In one sentence, why — and what fixes it? (§5)

If those land, you're ready to turn 3D atoms into model inputs. Tell me when you want **Part 3** and I'll build it the same way: concept first, then code.